



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

School of Computer Science and Engineering (SCSE)

SC3020 Database System Principles

Project 1 - Design & Implementation of Database System

Lab Instructor : Prof Long Cheng

Tutorial Group : SCCL Group 24

Submitted on: 08 October 2023

Name	Matriculation Number	Contributions
SCSE EUGENE WEE JUN LIN	U2120698A	Data Storage, Design of B+ tree, Experiment 1 and 2
SCSE CLARE YEO SU QI	U2122615F	Deletion in B+ Tree, Experiment 5
SCSE GUO ZHI QI	U2120848E	Deletion in B+ Tree, Experiment 5
SCSE KHOO KAI LI, KELLY	U2121724G	Search in B+ Tree, Experiment 3 and 4

Table of Content

1. Overview	3
1.1. Introduction	3
1.2. Database Constraints	3
1.3. Our System Configuration	3
1.4. Technology used	3
2. Design of Database Systems	4
2.1. Storage Component Design	4
2.1.1. Design #1: Records Structure	4
2.1.2. Design #2: Block Structure	6
2.1.3. Design #3: Database File Structure	7
2.2. Indexing Component Design	8
2.2.1. Design #1: Structure of B+ Tree Node	8
2.2.2. Design #2: Fixed Size B+ Tree Block	9
2.2.3. Design #3: Structure of B+ Tree	10
2.2.4. Implementation #1: Reading of Data Records	10
2.2.5. Implementation #2: Construction of the B+ Tree	11
2.2.6. Implementation #3: Search	13
2.2.7. Implementation #4: Deletion	17
3. Experiments on Database System	20
3.1. Overview	20
3.2. Experiment 1: Data Storage	20
3.3. Experiment 2: Building B+ Tree	21
3.4. Experiment 3 & 4: Database Queries	22
3.5. Experiment 5: Records Deletion	24
4. Conclusion	25
5. Appendices	26
5.1. Project Setup	26
5.2. Helper Functions	27
Memory Pool	27
B+ Tree Insertion	28
B+ Tree Search	29
B+ Tree Deletion	31

1. Overview

1.1. Introduction

This project serves to design and implement the following two components of a database management system, **storage and indexing**. This project is part of SC3020 Database System Principles.

1.2. Database Constraints

For the **storage component**, the following settings are assumed:

- A fraction of Main Memory is allocated for disk storage.
- Disk capacity limited to 100 - 500 MB.
- Disk storage is organised and accessed with a block as a unit.
- The block size is set to be 400 B.

For the **indexing component**, the following settings are assumed:

- B+ tree is used, which is stored in Main Memory, and each node is stored in a memory region bounded by the block size.
- Ability to support data with duplicates of keys.

1.3. Our System Configuration

For this project, our team utilised the following machine's main memory configuration.

1.4. Technology used

For this project, our team decided to implement the database system in C++, as recommended as it allows for fine-grained control over low-level memory management, which is required for database implementations.



Figure 1: Main Memory Configuration

2. Design of Database Systems

2.1. Storage Component Design

2.1.1. Design #1: Records Structure

For C++ language, the smallest group of bits is **unsigned char**, which is a group of 8 bits. Hence, when designing our records structure, addresses can only begin in **multiples of 8 bits (1 byte)**.

The record structure is defined in the following table:

Variable Name (Stored Variable)	Example	Data Type	Number of bits	
			Required	Allocated
Block Address	0x12345678	void*	64	64 (8 bytes)
GAME_DATE_EST (bin_game_date)	22/12/2022	uint32_t	24	32 (4 bytes)
TEAM_ID_home (team_id_home)	1610612740	uint32_t	32	32 (4 bytes)
PTS_home (pts_home)	126	uint16_t	10	16 (2 bytes)
FG_PCT_home (fg_pct_home)	0.484	float	32	32 (4 bytes)
FT_PCT_home (ft_pct_home)	0.926	float	32	32 (4 bytes)
FG3_PCT_home (pg3_pct_home)	0.382	float	32	32 (4 bytes)
AST_home (ast_home)	25	uint16_t	10	16 (2 byte)
REB_home (reb_home)	46	uint16_t	10	16 (2 byte)
HOME_TEAM_WINS (home_team_wins)	1	uint16_t	2	16 (2 byte)
Total bits allocated:	288 bits (320 bits total) [36 bytes + 4 bytes padding]			

Table 2: Breakdown of Record structure, by bits

To optimise our database memory storage, we utilised the following methodologies in our database storage design.

Optimised variable memory allocation

Firstly, we opted for data types which **utilise the least amount of memory** to store each field in our memory pool. For instance, there are variables that do not require the entire `int` format (4 bytes) to store data. Such variables either have a **fixed range** like `AST_home`, or are **binary in nature**, `home_team_wins`. Hence, we utilised the appropriate, yet minimum required memory for such variables (refer to Table 1).

Bitwise Operation to store date

Secondly, for the `date` column, rather than storing it as a string (24 bytes) which is not space efficient, our team opted to utilise bitwise operation to store date as an `uint32_t` data format. Albeit, processing the date requires bit-masking which is relatively more complex, but it **reduces the memory space significantly to 4 bytes**.

Fixed-length variables

For this assignment, the variables in the dataset are independent of the user's input, as the project does not consider other data and hence **format of records remains consistent** during the file reading process. Therefore, we can utilise fixed-length for each variable in the record.

Missing data in records

During our data analysis, we observed that 99 data records had missing information for key variables (i.e. `FG_PCT_home` and `FG3_PCT_home`). We decided to omit these missing values when storing into the disk.

```
----- DATA READING COMPLETE -----  
Summary of Memory Pool:  
2.06223MB / 100MB  
Missing data: 99
```

Figure 3: Count of Missing data records

2.1.2. Design #2: Block Structure

For our block structure, our team included a **block header** which contains information about the current block, such as number of records within the block (`num_records`) and memory size (`size`). The **contents of the block** are stored in the **remaining memory space**, excluding the block header (Figure 4).

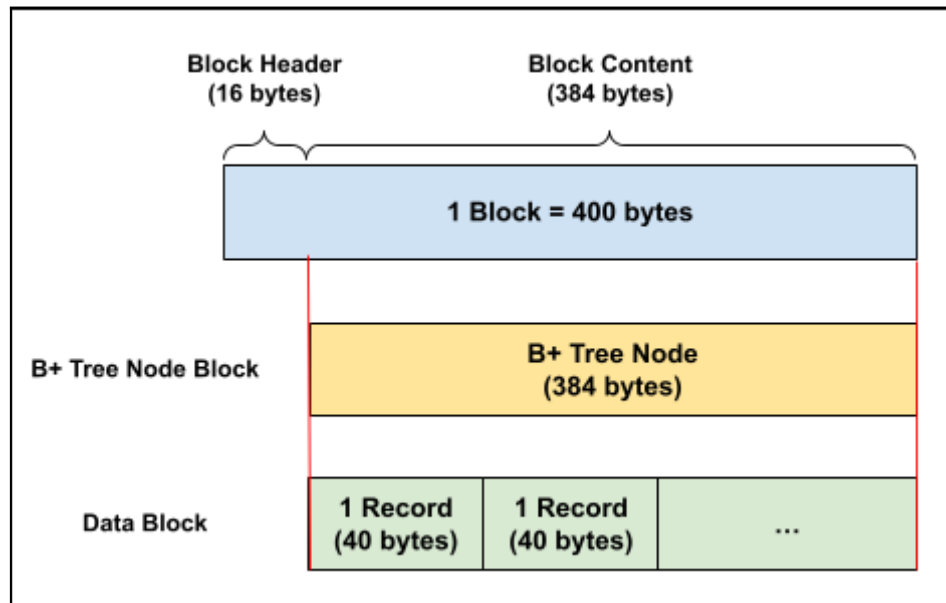


Figure 4: Block Structure

Block Types

We also segregated the block into 2 different types, **Data Block** and **B+ Tree Node Block**, where the contents are suited based on its functionality, storing data records and B+ tree node respectively.

Unspanned & Sequenced blocks

Since our **record size is smaller than the specified block size**, we utilised an unspanned block structure for **simplicity** during implementation. Data records within the block are **stored contiguously**, allowing for **more efficient data accesses** since they are read sequentially. This also **avoids unnecessary memory overhead** required for block management, in contrast to spanned block structure.

2.1.3. Design #3: Database File Structure

For our database file, we allocated a fixed space in our main memory of 100MB, serving as the memory pool and incrementally added blocks to the pool **as and when necessary** (Figure 5). This allows us to optimise memory usage, and utilise them when required.

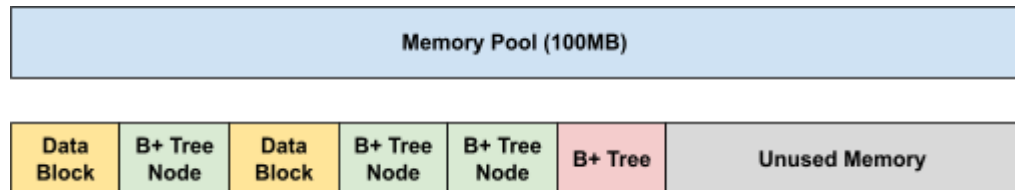


Figure 5: Memory Pool in Database file

Memory Pool Utilisation

Our approach involves initialising a memory space using `memset()` and `char[100*1024*124]` for a fixed-size memory pool of 100MB capacity.

We save the contents of the block to the disk by **allocating a memory space** within the memory pool and **retrieving the memory address**, `void*` then subsequently using the `memcpy()` function to **transfer contents** of the data record and B+ tree node to their respective blocks in the memory pool.

Once all the data records and B+ tree nodes have been stored in the memory pool, we will allocate a block to store the B+ tree so that we can access it once it is stored in memory.

Simulating Disk Access

For our project, we simulated the process of **storing each line of record into the disk** (memory pool) in main memory using a while loop. then subsequently accessing the B+ tree by **reading the B+ tree block**. Using the B+ tree block, we are then able to access the respective B+ tree nodes and data blocks.

2.2. Indexing Component Design

For our B+ tree design and implementation, our team adhered to the following rules.

2.2.1. Design #1: Structure of B+ Tree Node

Our constraints for this database implementation were two-folds, adhering to the **fixed block size of 400B** and to **support duplicate keys**.

To support duplicate keys, our team created a **unique key** built using a combination of both “FG_PCT_home” key value and count of duplicated key values, the structure is as given in Figure 6. Each B+ tree key has a **size of 8 bytes** (4 bytes from float, 4 bytes from int).

```
struct BPlusTreeKey {  
    float key; // Stores the value of FG_PCT_home  
    int count; // Stores the # of instance of the key  
};
```

Figure 6: B+ Tree key structure

For flexibility during implementation, we included a pointer to the next B+ tree node, which is useful during B+ leaf node accesses.

```
class BPlusTreeNode {  
    bool is_leaf;  
    int size;  
    BPlusTreeNode* next; // Stores the next BPlusTree node  
    BPlusTreeNode* parent; // Stores the parent BPlusTree node  
  
    BPlusTreeKey keys[m-1];  
    void* children[m];  
};
```

Figure 7: B+ Tree node structure

2.2.2. Design #2: Fixed Size B+ Tree Block

Each B+ tree node is represented using a fixed-size block in the memory pool. We would have to factor in the memory size constraint of 400B, alongside the need for a block header.

$$n = \frac{\text{block size} - \text{block header} - \text{B+ node overhead}}{\text{pointer size} + \text{B+ key size}}$$

block size = 400 bytes

block header = 16 bytes

B+ node overhead = 32 bytes (i.e. `is_leaf`, `size`, `next`, `parent`)

pointer size = 8 (void pointer)*

B+ key size = 8 bytes

Using this formula, we can compute the value of m which represents the maximum number of keys per B+ tree node. Thus, we derive 22 keys per node.

For block size = 400 bytes,

$$n = \frac{400 - 16 - 32}{8 + 8} = 22$$

Using the value of m = 22, our B+ tree node satisfies the memory size constraint of 400B, as seen in Figure 8.

```
class BPlusTreeNode {
    bool is_leaf;
    int size;
    BPlusTreeNode* next;
    BPlusTreeNode* parent;
    BPlusTreeNode* children[22];
    void* children_ptr;
};
```

Declared in: BPlusTree.h

class BPlusTreeNode

Size = 400 bytes

Copy: construction, assignment

Move: construction, assignment

⋮

Figure 8: B+ Tree node fixed at 400 bytes

2.2.3. Design #3: Structure of B+ Tree

For our B+ tree, we followed the same structure as specified in the lecture, where the **non-leaf nodes are sparse indices** and the **leaf node layer is a dense index**, corresponding to the B+ tree and data record block respectively. Hence, in order to access the desired data record, we would have to traverse the B+ tree via the non-leaf nodes, then access the data records in the leaf nodes.

2.2.4. Implementation #1: Reading of Data Records

We initialise a fixed-size memory pool of 100MB, and read the data records saved in the txt file **sequentially**. We stored the data using a **Record** structure and allocated a memory space for the record within the memory pool, which makes up **record_address**. This is subsequently used during the B+ tree construction below.

```
Allocating 100MB for memory pool

Reading data from text files...

First data record stored at 0x14e000010
BPlusTree stored at 0x14e20de20

----- DATA READING COMPLETE -----
Summary of Memory Pool:
2.0546MB / 100MB
Missing data: 99
```

Figure 9: Result of data storage

From Figure 9, we returned the memory address of the data record and the B+ tree for checking. We also utilised a total of 2.06MB of memory space within our memory pool, which accounts for both the data blocks and B+ tree node blocks.

2.2.5. Implementation #2: Construction of the B+ Tree

For B+ tree key insertion, we abstracted the main key insertion function into the following sub-functions, refer to Appendix 6.1 for more information. We mainly used a recursive function for upward propagation of the B+ tree key and followed the pseudocode provided in the lecture.

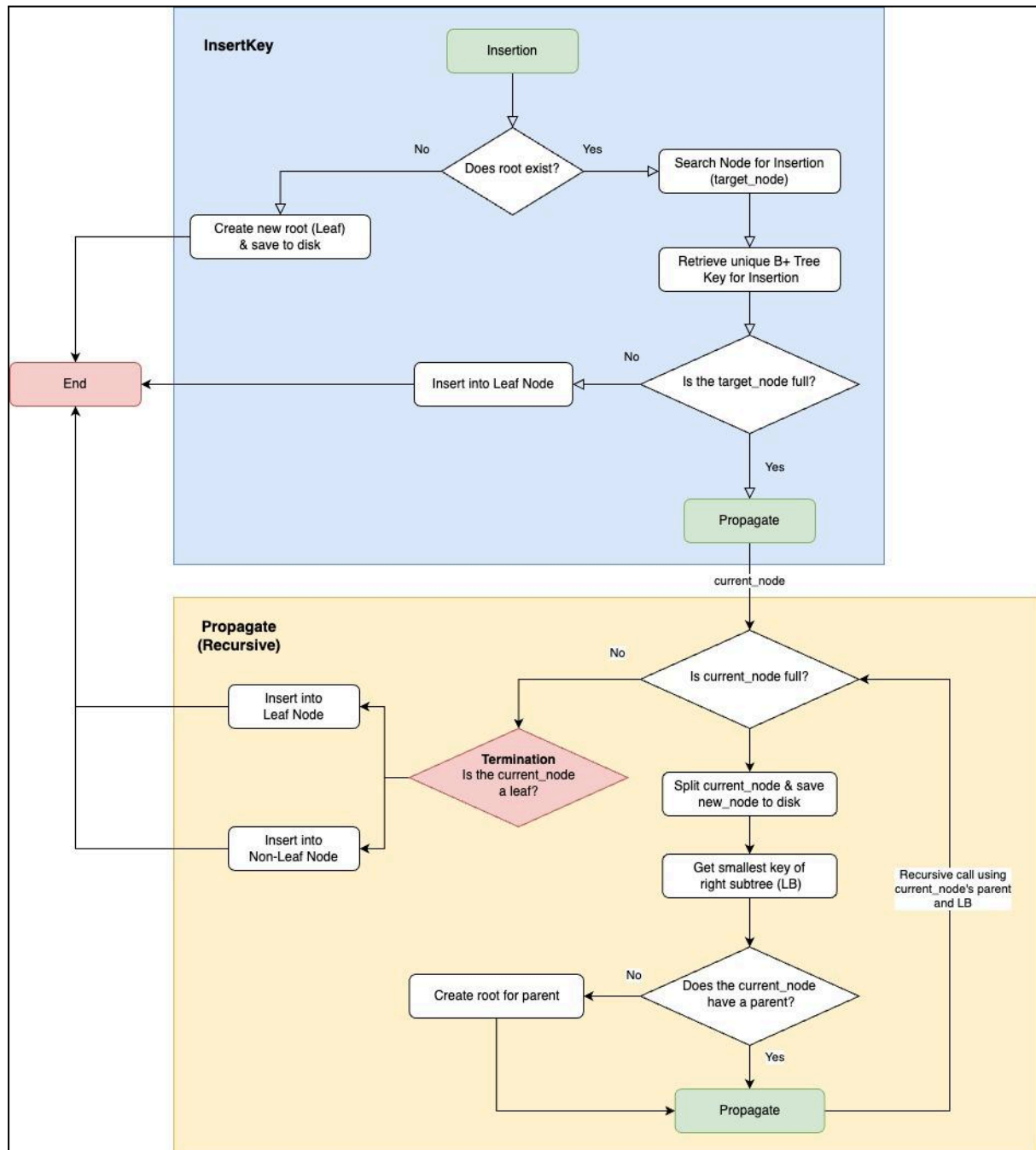


Figure 10: Insertion of B+ Tree Key Diagram

Recursive Upward Propagation

For our recursive call for upward propagation, the logic is to update the parent nodes based on the newly inserted key in the B+ tree.

This occurs whenever the **node for insertion is at its full capacity** ($m = 22$) and there is a need to split the node to accommodate the new B+ tree key.

Furthermore, we would need to **update the lower bound of the right subtree of the parent nodes**, which we handled in the `insertIntoNonLeafNode` function. This function inserts and sorts the non leaf node keys in ascending order, maintaining the integrity of the B+ tree.

Result of B+ Tree Construction

Figure 11 shows the result of our B+ tree when displayed sequentially, where Level 0 is the root node, and the n^{th} level is the leaf node. Intermediate levels represent the internal nodes of the B+ tree.

```
Level 0: (0.402, 16) (0.426, 2) (0.444, 6) (0.463, 3) (0.481, 36) (0.5, 135) (0.524, 23) | Size: 7
Level 1: (0.33, 1) (0.345, 7) (0.354, 1) (0.36, 4) (0.364, 25) (0.368, 5) (0.371, 25) (0.375, 4) (0.377, 13) (0.379, 14) (0.382, 8) (0.384, 12) (0.386, 1) (0.388, 1) (0.39, 1) (0.392, 1) (0.394, 1) (0.396, 1) (0.398, 1) (0.4, 1) (0.402, 1) (0.404, 1) (0.406, 1) (0.408, 1) (0.41, 1) (0.412, 1) (0.414, 1) (0.416, 1) (0.418, 1) (0.42, 1) (0.422, 1) (0.424, 1) (0.426, 1) (0.428, 1) (0.43, 1) (0.432, 1) (0.434, 1) (0.436, 1) (0.438, 1) (0.44, 1) (0.442, 1) (0.444, 1) (0.446, 1) (0.448, 1) (0.45, 1) (0.452, 1) (0.454, 1) (0.456, 1) (0.458, 1) (0.46, 1) (0.462, 1) (0.464, 1) (0.466, 1) (0.468, 1) (0.47, 1) (0.472, 1) (0.474, 1) (0.476, 1) (0.478, 1) (0.48, 1) (0.482, 1) (0.484, 1) (0.486, 1) (0.488, 1) (0.49, 1) (0.492, 1) (0.494, 1) (0.496, 1) (0.498, 1) (0.5, 1) (0.502, 1) (0.504, 1) (0.506, 1) (0.508, 1) (0.51, 1) (0.512, 1) (0.514, 1) (0.516, 1) (0.518, 1) (0.52, 1) (0.522, 1) (0.524, 1) (0.526, 1) (0.528, 1) (0.53, 1) (0.532, 1) (0.534, 1) (0.536, 1) (0.538, 1) (0.54, 1) (0.542, 1) (0.544, 1) (0.546, 1) (0.548, 1) (0.55, 1) (0.552, 1) (0.554, 1) (0.556, 1) (0.558, 1) (0.56, 1) (0.562, 1) (0.564, 1) (0.566, 1) (0.568, 1) (0.57, 1) (0.572, 1) (0.574, 1) (0.576, 1) (0.578, 1) (0.58, 1) (0.582, 1) (0.584, 1) (0.586, 1) (0.588, 1) (0.59, 1) (0.592, 1) (0.594, 1) (0.596, 1) (0.598, 1) (0.6, 1) (0.602, 1) (0.604, 1) (0.606, 1) (0.608, 1) (0.61, 1) (0.612, 1) (0.614, 1) (0.616, 1) (0.618, 1) (0.62, 1) (0.622, 1) (0.624, 1) (0.626, 1) (0.628, 1) (0.63, 1) (0.632, 1) (0.634, 1) (0.636, 1) (0.638, 1) (0.64, 1) (0.642, 1) (0.644, 1) (0.646, 1) (0.648, 1) (0.65, 1) (0.652, 1) (0.654, 1) (0.656, 1) (0.658, 1) (0.66, 1) (0.662, 1) (0.664, 1) (0.666, 1) (0.668, 1) (0.67, 1) (0.672, 1) (0.674, 1) (0.676, 1) (0.678, 1) (0.68, 1) (0.682, 1) (0.684, 1) (0.686, 1) (0.688, 1) (0.69, 1) (0.692, 1) (0.694, 1) (0.696, 1) (0.698, 1) (0.7, 1) (0.702, 1) (0.704, 1) (0.706, 1) (0.708, 1) (0.71, 1) (0.712, 1) (0.714, 1) (0.716, 1) (0.718, 1) (0.72, 1) (0.722, 1) (0.724, 1) (0.726, 1) (0.728, 1) (0.73, 1) (0.732, 1) (0.734, 1) (0.736, 1) (0.738, 1) (0.74, 1) (0.742, 1) (0.744, 1) (0.746, 1) (0.748, 1) (0.75, 1) (0.752, 1) (0.754, 1) (0.756, 1) (0.758, 1) (0.76, 1) (0.762, 1) (0.764, 1) (0.766, 1) (0.768, 1) (0.77, 1) (0.772, 1) (0.774, 1) (0.776, 1) (0.778, 1) (0.78, 1) (0.782, 1) (0.784, 1) (0.786, 1) (0.788, 1) (0.79, 1) (0.792, 1) (0.794, 1) (0.796, 1) (0.798, 1) (0.8, 1) (0.802, 1) (0.804, 1) (0.806, 1) (0.808, 1) (0.81, 1) (0.812, 1) (0.814, 1) (0.816, 1) (0.818, 1) (0.82, 1) (0.822, 1) (0.824, 1) (0.826, 1) (0.828, 1) (0.83, 1) (0.832, 1) (0.834, 1) (0.836, 1) (0.838, 1) (0.84, 1) (0.842, 1) (0.844, 1) (0.846, 1) (0.848, 1) (0.85, 1) (0.852, 1) (0.854, 1) (0.856, 1) (0.858, 1) (0.86, 1) (0.862, 1) (0.864, 1) (0.866, 1) (0.868, 1) (0.87, 1) (0.872, 1) (0.874, 1) (0.876, 1) (0.878, 1) (0.88, 1) (0.882, 1) (0.884, 1) (0.886, 1) (0.888, 1) (0.89, 1) (0.892, 1) (0.894, 1) (0.896, 1) (0.898, 1) (0.9, 1) (0.902, 1) (0.904, 1) (0.906, 1) (0.908, 1) (0.91, 1) (0.912, 1) (0.914, 1) (0.916, 1) (0.918, 1) (0.92, 1) (0.922, 1) (0.924, 1) (0.926, 1) (0.928, 1) (0.93, 1) (0.932, 1) (0.934, 1) (0.936, 1) (0.938, 1) (0.94, 1) (0.942, 1) (0.944, 1) (0.946, 1) (0.948, 1) (0.95, 1) (0.952, 1) (0.954, 1) (0.956, 1) (0.958, 1) (0.96, 1) (0.962, 1) (0.964, 1) (0.966, 1) (0.968, 1) (0.97, 1) (0.972, 1) (0.974, 1) (0.976, 1) (0.978, 1) (0.98, 1) (0.982, 1) (0.984, 1) (0.986, 1) (0.988, 1) (0.99, 1) (0.992, 1) (0.994, 1) (0.996, 1) (0.998, 1) (1, 1)
Level 2: (0.279, 2) (0.295, 2) (0.303, 1) (0.305, 3) (0.309, 1) (0.312, 2) (0.314, 2) (0.316, 1) (0.319, 3) (0.321, 2) (0.323, 2) (0.325, 4) (0.327, 2) (0.329, 2) (0.331, 2) (0.333, 2) (0.335, 2) (0.337, 2) (0.339, 2) (0.341, 2) (0.343, 2) (0.345, 2) (0.347, 2) (0.349, 2) (0.351, 2) (0.353, 2) (0.355, 2) (0.357, 2) (0.359, 2) (0.361, 2) (0.363, 2) (0.365, 2) (0.367, 2) (0.369, 2) (0.371, 2) (0.373, 2) (0.375, 2) (0.377, 2) (0.379, 2) (0.381, 2) (0.383, 2) (0.385, 2) (0.387, 2) (0.389, 2) (0.391, 2) (0.393, 2) (0.395, 2) (0.397, 2) (0.399, 2) (0.401, 2) (0.403, 2) (0.405, 2) (0.407, 2) (0.409, 2) (0.411, 2) (0.413, 2) (0.415, 2) (0.417, 2) (0.419, 2) (0.421, 2) (0.423, 2) (0.425, 2) (0.427, 2) (0.429, 2) (0.431, 2) (0.433, 2) (0.435, 2) (0.437, 2) (0.439, 2) (0.441, 2) (0.443, 2) (0.445, 2) (0.447, 2) (0.449, 2) (0.451, 2) (0.453, 2) (0.455, 2) (0.457, 2) (0.459, 2) (0.461, 2) (0.463, 2) (0.465, 2) (0.467, 2) (0.469, 2) (0.471, 2) (0.473, 2) (0.475, 2) (0.477, 2) (0.479, 2) (0.481, 2) (0.483, 2) (0.485, 2) (0.487, 2) (0.489, 2) (0.491, 2) (0.493, 2) (0.495, 2) (0.497, 2) (0.499, 2) (0.501, 2) (0.503, 2) (0.505, 2) (0.507, 2) (0.509, 2) (0.511, 2) (0.513, 2) (0.515, 2) (0.517, 2) (0.519, 2) (0.521, 2) (0.523, 2) (0.525, 2) (0.527, 2) (0.529, 2) (0.531, 2) (0.533, 2) (0.535, 2) (0.537, 2) (0.539, 2) (0.541, 2) (0.543, 2) (0.545, 2) (0.547, 2) (0.549, 2) (0.551, 2) (0.553, 2) (0.555, 2) (0.557, 2) (0.559, 2) (0.561, 2) (0.563, 2) (0.565, 2) (0.567, 2) (0.569, 2) (0.571, 2) (0.573, 2) (0.575, 2) (0.577, 2) (0.579, 2) (0.581, 2) (0.583, 2) (0.585, 2) (0.587, 2) (0.589, 2) (0.591, 2) (0.593, 2) (0.595, 2) (0.597, 2) (0.599, 2) (0.601, 2) (0.603, 2) (0.605, 2) (0.607, 2) (0.609, 2) (0.611, 2) (0.613, 2) (0.615, 2) (0.617, 2) (0.619, 2) (0.621, 2) (0.623, 2) (0.625, 2) (0.627, 2) (0.629, 2) (0.631, 2) (0.633, 2) (0.635, 2) (0.637, 2) (0.639, 2) (0.641, 2) (0.643, 2) (0.645, 2) (0.647, 2) (0.649, 2) (0.651, 2) (0.653, 2) (0.655, 2) (0.657, 2) (0.659, 2) (0.661, 2) (0.663, 2) (0.665, 2) (0.667, 2) (0.669, 2) (0.671, 2) (0.673, 2) (0.675, 2) (0.677, 2) (0.679, 2) (0.681, 2) (0.683, 2) (0.685, 2) (0.687, 2) (0.689, 2) (0.691, 2) (0.693, 2) (0.695, 2) (0.697, 2) (0.699, 2) (0.701, 2) (0.703, 2) (0.705, 2) (0.707, 2) (0.709, 2) (0.711, 2) (0.713, 2) (0.715, 2) (0.717, 2) (0.719, 2) (0.721, 2) (0.723, 2) (0.725, 2) (0.727, 2) (0.729, 2) (0.731, 2) (0.733, 2) (0.735, 2) (0.737, 2) (0.739, 2) (0.741, 2) (0.743, 2) (0.745, 2) (0.747, 2) (0.749, 2) (0.751, 2) (0.753, 2) (0.755, 2) (0.757, 2) (0.759, 2) (0.761, 2) (0.763, 2) (0.765, 2) (0.767, 2) (0.769, 2) (0.771, 2) (0.773, 2) (0.775, 2) (0.777, 2) (0.779, 2) (0.781, 2) (0.783, 2) (0.785, 2) (0.787, 2) (0.789, 2) (0.791, 2) (0.793, 2) (0.795, 2) (0.797, 2) (0.799, 2) (0.801, 2) (0.803, 2) (0.805, 2) (0.807, 2) (0.809, 2) (0.811, 2) (0.813, 2) (0.815, 2) (0.817, 2) (0.819, 2) (0.821, 2) (0.823, 2) (0.825, 2) (0.827, 2) (0.829, 2) (0.831, 2) (0.833, 2) (0.835, 2) (0.837, 2) (0.839, 2) (0.841, 2) (0.843, 2) (0.845, 2) (0.847, 2) (0.849, 2) (0.851, 2) (0.853, 2) (0.855, 2) (0.857, 2) (0.859, 2) (0.861, 2) (0.863, 2) (0.865, 2) (0.867, 2) (0.869, 2) (0.871, 2) (0.873, 2) (0.875, 2) (0.877, 2) (0.879, 2) (0.881, 2) (0.883, 2) (0.885, 2) (0.887, 2) (0.889, 2) (0.891, 2) (0.893, 2) (0.895, 2) (0.897, 2) (0.899, 2) (0.901, 2) (0.903, 2) (0.905, 2) (0.907, 2) (0.909, 2) (0.911, 2) (0.913, 2) (0.915, 2) (0.917, 2) (0.919, 2) (0.921, 2) (0.923, 2) (0.925, 2) (0.927, 2) (0.929, 2) (0.931, 2) (0.933, 2) (0.935, 2) (0.937, 2) (0.939, 2) (0.941, 2) (0.943, 2) (0.945, 2) (0.947, 2) (0.949, 2) (0.951, 2) (0.953, 2) (0.955, 2) (0.957, 2) (0.959, 2) (0.961, 2) (0.963, 2) (0.965, 2) (0.967, 2) (0.969, 2) (0.971, 2) (0.973, 2) (0.975, 2) (0.977, 2) (0.979, 2) (0.981, 2) (0.983, 2) (0.985, 2) (0.987, 2) (0.989, 2) (0.991, 2) (0.993, 2) (0.995, 2) (0.997, 2) (0.999, 2) (1, 2)
Level 3: (0.25, 1) (0.257, 1) (0.266, 1) (0.269, 1) (0.269, 2) (0.274, 1) (0.275, 1) (0.277, 1) (0.277, 2) (0.278, 1) (0.279, 1) (0.279, 2) (0.282, 1) (0.282, 2) (0.283, 1) (0.284, 1) (0.286, 1) (0.288, 1) (0.288, 2) (0.289, 1) (0.289, 2) (0.29, 1) (0.291, 1) (0.292, 1) (0.293, 1) (0.295, 2) (0.295, 3) (0.296, 1) (0.297, 1) (0.298, 1) (0.298, 2) (0.298, 3) (0.298, 4) (0.298, 5) (0.299, 1) (0.3, 1) (0.301, 1) (0.301, 2) (0.303, 1) (0.303, 2) (0.303, 3) (0.303, 4) (0.303, 5) (0.304, 1) (0.304, 2) (0.304, 3) (0.304, 4) (0.305, 1) (0.305, 2) (0.305, 3) (0.305, 4) (0.305, 5) (0.306, 1) (0.306, 2) (0.306, 3) (0.306, 4) (0.306, 5) (0.307, 1) (0.307, 2) (0.307, 3) (0.307, 4) (0.307, 5) (0.308, 1) (0.308, 2) (0.308, 3) (0.308, 4) (0.308, 5) (0.309, 1) (0.309, 2) (0.309, 3) (0.309, 4) (0.309, 5) (0.31, 1) (0.31, 2) (0.31, 3) (0.31, 4) (0.31, 5) (0.31, 6) (0.31, 7) (0.311, 1) (0.311, 2) (0.311, 3) (0.311, 4) (0.311, 5) (0.311, 6) (0.311, 7) (0.311, 8) (0.311, 9) (0.311, 10) (0.311, 11) (0.311, 12) (0.311, 13) (0.311, 14) (0.311, 15) (0.311, 16) (0.311, 17) (0.311, 18) (0.311, 19) (0.311, 20) (0.311, 21) (0.311, 22) (0.311, 23) (0.311, 24) (0.311, 25) (0.311, 26) (0.311, 27) (0.311, 28) (0.311, 29) (0.311, 30) (0.311, 31) (0.311, 32) (0.311, 33) (0.311, 34) (0.311, 35) (0.311, 36) (0.311, 37) (0.311, 38) (0.311, 39) (0.311, 40) (0.311, 41) (0.311, 42) (0.311, 43) (0.311, 44) (0.311, 45) (0.311, 46) (0.311, 47) (0.311, 48) (0.311, 49) (0.311, 50) (0.311, 51) (0.311, 52) (0.311, 53) (0.311, 54) (0.311, 55) (0.311, 56) (0.311, 57) (0.311, 58) (0.311, 59) (0.311, 60) (0.311, 61) (0.311, 62) (0.311, 63) (0.311, 64) (0.311, 65) (0.311, 66) (0.311, 67) (0.311, 68) (0.311, 69) (0.311, 70) (0.311, 71) (0.311, 72) (0.311, 73) (0.311, 74) (0.311, 75) (0.311, 76) (0.311, 77) (0.311, 78) (0.311, 79) (0.311, 80) (0.311, 81) (0.311, 82) (0.311, 83) (0.311, 84) (0.311, 85) (0.311, 86) (0.311, 87) (0.311, 88) (0.311, 89) (0.311, 90) (0.311, 91) (0.311, 92) (0.311, 93) (0.311, 94) (0.311, 95) (0.311, 96) (0.311, 97) (0.311, 98) (0.311, 99) (0.311, 100) (0.311, 101) (0.311, 102) (0.311, 103) (0.311, 104) (0.311, 105) (0.311, 106) (0.311, 107) (0.311, 108) (0.311, 109) (0.311, 110) (0.311, 111) (0.311, 112) (0.311, 113) (0.311, 114) (0.311, 115) (0.311, 116) (0.311, 117) (0.311, 118) (0.311, 119) (0.311, 120) (0.311, 121) (0.311, 122) (0.311, 123) (0.311, 124) (0.311, 125) (0.311, 126) (0.311, 127) (0.311, 128) (0.311, 129) (0.311, 130) (0.311, 131) (0.311, 132) (0.311, 133) (0.311, 134) (0.311, 135) (0.311, 136) (0.311, 137) (0.311, 138) (0.311, 139) (0.311, 140) (0.311, 141) (0.311, 142) (0.311, 143) (0.311, 144) (0.311, 145) (0.311, 146) (0.311, 147) (0.311, 148) (0.311, 149) (0.311, 150) (0.311, 151) (0.311, 152) (0.311, 153) (0.311, 154) (0.311, 155) (0.311, 156) (0.311, 157) (0.311, 158) (0.311, 159) (0.311, 160) (0.311, 161) (0.311, 162) (0.311, 163) (0.311, 164) (0.311, 165) (0.311, 166) (0.311, 167) (0.311, 168) (0.311, 169) (0.311, 170) (0.311, 171) (0.311, 172) (0.311, 173) (0.311, 174) (0.311, 175) (0.311, 176) (0.311, 177) (0.311, 178) (0.311, 179) (0.311, 180) (0.311, 181) (0.311, 182) (0.311, 183) (0.311, 184) (0.311, 185) (0.311, 186) (0.311, 187) (0.311, 188) (0.311, 189) (0.311, 190) (0.311, 191) (0.311, 192) (0.311, 193) (0.311, 194) (0.311, 195) (0.311, 196) (0.311, 197) (0.311, 198) (0.311, 199) (0.311, 200) (0.311, 201) (0.311, 202) (0.311, 203) (0.311, 204) (0.311, 205) (0.311, 206) (0.311, 207) (0.311, 208) (0.311, 209) (0.311, 210) (0.311, 211) (0.311, 212) (0.311, 213) (0.311, 214) (0.311, 215) (0.311, 216) (0.311, 217) (0.311, 218) (0.311, 219) (0.311, 220) (0.311, 221) (0.311, 222) (0.311, 223) (0.311, 224) (0.311, 225) (0.311, 226) (0.311, 227) (0.311, 228) (0.311, 229) (0.311, 230) (0.311, 231) (0.311, 232) (0.311, 233) (0.311, 234) (0.311, 235) (0.311, 236) (0.311, 237) (0.311, 238) (0.311, 239) (0.311, 240) (0.311, 241) (0.311, 242) (0.311, 243) (0.311, 244) (0.311, 245) (0.311, 246) (0.311, 247) (0.311, 248) (0.311, 249) (0.311, 250) (0.311, 251) (0.311, 252) (0.311, 253) (0.311, 254) (0.311, 255) (0.311, 256) (0.311, 257) (0.311, 258) (0.311, 259) (0.311, 260) (0.311, 261) (0.311, 262) (0.311, 263) (0.311, 264) (0.311, 265) (0.311, 266) (0.311, 267) (0.311, 268) (0.311, 269) (0.311, 270) (0.311, 271) (0.311, 272) (0.311, 273) (0.311, 274) (0.311, 275) (0.311, 276) (0.311, 277) (0.311, 278) (0.311, 279) (0.311, 280) (0.311, 281) (0.311, 282) (0.311, 283) (0.311, 284) (0.311, 285) (0.311, 286) (0.311, 287) (0.311, 288) (0.311, 289) (0.311, 290) (0.311, 291) (0.311, 292) (0.311, 293) (0.311, 294) (0.311, 295) (0.311, 296) (0.311, 297) (0.311, 298) (0.311, 299) (0.311, 300) (0.311, 301) (0.311, 302) (0.311, 303) (0.311, 304) (0.311, 305) (0.311, 306) (0.311, 307) (0.311, 308) (0.311, 309) (0.311, 310) (0.311, 311) (0.311, 312) (0.311, 313) (0.311, 314) (0.311, 315) (0.311, 316) (0.311, 317) (0.311, 318) (0.311, 319) (0.311, 320) (0.311, 321) (0.311, 322) (0.311, 323) (0.311, 324) (0.311, 325) (0.311, 326) (0.311, 327) (0.311, 328) (0.311, 329) (0.311, 330) (0.311, 331) (0.311, 332) (0.311, 333) (0.311, 334) (0.311, 335) (0.311, 336) (0.311, 337) (0.311, 338) (0.311, 339) (0.311, 340) (0.311, 341) (0.311, 342) (0.311, 343) (0.311, 344) (0.311, 345) (0.311, 346) (0.311, 347) (0.311, 348) (0.311, 349) (0.311, 350) (0.311, 351) (0.311, 352) (0.311, 353) (0.311, 354) (0.311, 355) (0.311, 356) (0.311, 357) (0.311, 358) (0.311, 359) (0.311, 360) (0.311, 361) (0.311, 362) (0.311, 363) (0.311, 364) (0.311, 365) (0.311, 366) (0.311, 367) (0.311, 368) (0.311, 3
```

2.2.6. Implementation #3: Search

searchKey

```

void BPlusTree::searchKey(MemoryPool *disk, float lowerkey, float upperkey)
{
    int indexblks = 0;

    SearchAddresslist.clear();

    // If tree is empty
    if (root == nullptr)
    {
        cout << "B+ Tree is empty\n";
    }
    else
    {
        auto current_node = (BPlusTreeNode *) root;
        while (!current_node->is_leaf) {
            for (int i = 0; i < current_node->size; i++) {
                if (current_node->is_leaf) {
                    break;
                }
                indexblks++;
            }

            // If key to be inserted is lesser than current node key, go to the left
            children
            if (current_node->keys[i].key >= lowerkey) {
                current_node = (BPlusTreeNode *) current_node->children[i];
                break;
            }
            // Key to be inserted is greater than or equal to current node key
            else {
                // Terminate and enter to the right-most children if all exhausted
                if (i == current_node->size-1) {
                    current_node = (BPlusTreeNode *) current_node->children[i+1];
                    break;
                }
                // Iterate to next key
                continue;
            }
        }

        int count = 0;
        while (current_node->keys[0].key <= upperkey)
        {
            for (int i = 0; i < current_node->size; i++)
            {
                if (current_node->keys[i].key >= lowerkey &&
                    current_node->keys[i].key <= upperkey)
                {
                    auto temp_address = current_node->children[i];
                    SearchAddresslist.push_back(temp_address);
                    disk->displayRecord(temp_address);
                    count++;
                }
            }

            if (current_node->next != nullptr)
            {
                current_node = current_node->next;
            }
            else
            {
                break;
            }
        }

        cout << " - Number of records retrieved: " << count << endl;
    }
}

```

1. Check if there is no root node which means the tree is empty
2. Else while current node is not leaf, iterate through each key in the node and check whether the key is greater than or equal to lower bound of "FG_PCT_home" range. If yes, then go to the left child
3. If lower bound of range is greater than all keys in node, go to the right most child
4. Count the number of index nodes accessed and store in indexblks
5. Repeat steps 2 and 3 until current node reaches leaf node level and gives the leaf node that contains first instance of lower bound key

6. Iterate through each key in the leaf node and check if the key is within range. If yes, retrieve the record address from the children node pointed by the key and store it in global vector SearchAddresslist.
7. If pointer to next leaf node is null, then break. Else move to the next leaf node and repeat steps 5 and 6

getAverage

```
double BPlusTree::getAverage(MemoryPool *disk)
{
    double sum=0;
    double average=0;
    double value;

    for (int i=0; i < SearchAddresslist.size(); i++)
    {
        value = disk->loadRecordfcg3(SearchAddresslist[i]) ;
        sum = sum + value ;
    }

    if (SearchAddresslist.size() != 0)
    {
        average = sum / SearchAddresslist.size();
        return average;
    }
}
```

1. Iterate through size of SearchAddresslist that stores address of each record and call function loadRecordfcg3 for each record address to get value "FG3_PCT_home"
2. Sum up all the values and divide by size of list to find average
3. Return average

getNumDataBlock

```
int BPlusTree::getNumDataBlock(MemoryPool *disk)
{
    void* ID1;
    vector<void*> IdArray;
    int count = 0;
    int j;

    IdArray.clear();

    for (int i = 0; i < SearchAddresslist.size(); i++)
    {
        ID1 = disk->getBlockAddress(SearchAddresslist[i]);
        IdArray.push_back(ID1);
    }

    for (int i = 1; i < IdArray.size(); i++)
    {
        for (j = 0; j < i; j++)
        {
            if (IdArray[i] == IdArray[j])
            {
                break;
            }
        }

        if (i==j)
        {
            count++;
        }
    }

    return count;
}
```

1. Iterate through size of SearchAddresslist that stores address of each record and call function getBlockAddress() for each record address to get data block address of where record is stored.
2. Store block address in array
3. Iterate through all block addresses in array to find count of distinct addresses as each data block can store multiple data records and return count

getBlocksAccessedByBruteForce

```
int MemoryPool::getBlocksAccessedByBruteForce(float lowerkey, float upperkey)
{
    int datablks = 0;

    BFSearchAddresslist.clear() ;

    for (int i = 0; i < block_ptr_list.size(); i++)
    {
        int num_records = 0;
        int used_size = 0;

        while (used_size < block_size)
        {
            auto record = new Record();
            void* record_ptr = (void*)((char*) block_ptr_list[i] + num_records *
RECORD_SIZE);

            if (record != nullptr)
            {
                record = loadRecord(record_ptr);
            }

            else
            {
                break;
            }

            float value = record->fg_pct_home;

            if (value >= lowerkey && value <= upperkey)
            {
                BFSearchAddresslist.push_back(record_ptr);
            }

            used_size = used_size + RECORD_SIZE;
            num_records++;
        }

        datablks++;
    }

    return datablks;
}
```

1. Iterate through all data blocks from `block_ptr_list` that stores the starting address of each block of data records
2. For each data block, iterate through all records stored in the block by finding address of record that is set equal to `block_ptr_list[i] + num_records * RECORD_SIZE` where `num_records` is a counter that keeps track of the number of records iterated within the block.
3. If record address doesn't point to null pointer, load the record and find value of "FG_PCT_home"
4. If value is within range, store record address in vector of AddressList
5. Increment data blocks accessed and return value

2.2.7. Implementation #4: Deletion

deleteKey() Algorithm

- Our algorithm uses a recursive approach. From the root node, the algorithm first recursively transverse down to the leaf node where deleteKey is performed. Each node will then return its result to its parent. At every non leaf node, it iterates through all its children nodes and performs deleteKey iteratively, updates the current non leaf node and returns its result up to the parent recursively.
- At the **leaf node**, there are 2 main cases:
 - Full deletion of leaf node (Returns 1)
 - Occurs when the last key of the leaf node is smaller than target key, meaning all keys in the leaf node are all smaller than target key and thus all deleted
 - Occurs when left node keys merges with the keys in right sibling node (when right sibling node has insufficient keys to borrow) , resulting in deletion of left leaf node
 - Partial deletion of leaf node (Returns 0)
 - Occurs when some keys are larger than target key
 - Remaining keys in the nodes will need to satisfy the requirement of minimum $\lfloor (m+1)/2 \rfloor$ keys
 - borrowing keys from its right sibling node to maintain its minimum number keys criteria (when right node has sufficient keys to borrow)
 - Merging in to the right node if there is insufficient keys to borrow
- For every deletion in leaf node, the corresponding record in disk is deleted as well
- When Borrowing from Right or Merging with Right, the right node keys will be updated. Thus, checkKey() will be called on the right sibling node, to recursively check that the corresponding key in each parent is properly updated with the lower bound of the right subtree.

- At a **non leaf node**,
 - The result of deleteKey() in its child node is returned to the deleteKey() function in the parent non leaf node.
 - If the entire child node is deleted (returned 1) , the corresponding key in parent node is deleted as well
 - If the child node is only partially deleted (returned 0), checkKey is called on the child node, to ensure that the corresponding keys in each recursive parent is updated as well
 - After deletion in the Non leaf node,
 - Case 1 : Non leaf node is left with insufficient keys ($< \lfloor m/2 \rfloor$ keys)
 - It will borrow or merge from its right non leaf node. This is similar to the case mentioned above in leaf nodes.
 - If borrow, the functions returns 0 to parent, and if it merged with right node, the non leaf node is deleted and it returns 1 to the parent
 - Case 2 : Non leaf node has sufficient keys left ($\geq \lfloor m/2 \rfloor$ keys)
 - CheckKey() is performed in current node to ensure keys in its' parents are recursively updated
 - Case 3: Non leaf node has 0 keys and no child node
 - Entire node is deleted and returns 1 to parent
 - This is performed recursively upwards until the root node
 - If the non leaf node is a root node, it does not need to satisfy the minimum keys requirement. If there are 0 keys and 1 child node, the new root will be the child node.

19

3. Experiments on Database System

3.1. Overview

This section serves to document the results of the following experiments based on our design and implementation of the database system specified in Section 2.

3.2. Experiment 1: Data Storage

This section reports the findings for **Experiment 1**:

store the data (which is about NBA games) on the disk (as specified in Section 1.2 Database Constraints)

```
Experiment 1: Reading data text file into DB system.  
- Number of records: 26552  
- Size of a record: 40  
- Number of records stored in a block: 9  
- Number of blocks for storing the data: 2951
```

Figure 14: Experiment 1 results

Observation

We can see that the numbers tally up for the data storage where,

$$\# \text{ of data blocks} = \frac{\# \text{ of data records}}{\# \text{ of records per block}} = \frac{26552}{9} = 2950.12 \approx 2951 \text{ (rounded up)}$$

$$\# \text{ of records per block} = \frac{\text{block size} - \text{block header}}{\text{record size}} = \frac{400 - 16}{40} = 9.6 \approx 9 \text{ (rounded down)}$$

3.3. Experiment 2: Building B+ Tree

This section reports the findings for **Experiment 2**:

build a B+ tree on the attribute "FG_PCT_home" by inserting the records sequentially

```
Experiment 2: Build B+ Tree and report statistics
- Parameter of n of B+ tree: 22
- Number of nodes of the B+ tree: 2434
- Number of levels of the B+ tree: 4
- Content of root node: (0.402, 16) (0.426, 2) (0.444, 6) (0.463, 3) (0.481, 36) (0.5, 135) (0.524, 23)
```

Figure 15: Experiment 2 results

Observation

Theoretically, for our given B+ tree node design where $m = 22$ and each internal node is at least 50% utilised, at least $\frac{n}{2}$ keys (11) per node block and $\frac{n}{2} + 1$ children pointer (12).

For our given dataset with the number of records = 26,652, the **depth of the B+ tree should be between 4 to 5 levels**.

Level 3:

Maximum number of records that can be indexed = $22 * 23 * 23 = 11,638$

There are **insufficient records** for B+ tree with 3 levels.

Level 4:

Maximum number of records that can be indexed = $22 * 23 * 23 * 23 = 267,674$

Minimum number of records that can be indexed = $2 * 12 * 12 * 11 = 3,168$

For the maximum number of records, there are **sufficient records for 4 levels**, but albeit potentially **insufficient to accommodate the minimum number of records**.

Level 5:

Minimum number of records that can be indexed = $2 * 12 * 12 * 12 * 11 = 38,016$

For a B+ tree with 5 levels, there are **sufficient records** to accommodate all records for the minimum.

3.4. Experiment 3 & 4: Database Queries

This section reports the findings for **Experiment 3**:

retrieve those movies with the “FG_PCT_home” equal to 0.5

```
Experiment 3: Retrieve those movies with the attribute FG_PCT_home equal to 0.5
- Number of records retrieved: 848
- Running time of Retrieval process: 0.000155 seconds
- Number of index nodes the process accesses: 22
- Average of FG3_PCT_home values of the records that are returned: 0.39120047
- Number of data blocks the process accesses: 741
- Number of Data Blocks Accessed by Brute Force 2951
- Running time of Linear Scan Accessed by Brute Force : 0.001998209 seconds
```

Figure 16: Experiment 3 results

Observation

The time complexity of **searching a single value in B+ tree is $O(\log n + k)$** , where n is the number of nodes in the tree and k is the number of values in the range.

Search involves going through all levels of the tree until the leaf node with first instance of lower bound is found, hence it is $O(\log n)$, and the number of keys in the leaf nodes to iterate is proportional to number of values within the range even for searching single value, so time complexity is $O(k) + O(\log n)$.

The brute force approach, however, takes $O(m)$ time where m is the total of records, as it iterates through each record in each data block and checks for the desired key.

B+ tree running time of retrieval process is also **significantly faster than brute force approach** as brute force iterates through all data blocks to find whether each record in block is within range while B+ tree traverse through index nodes in each level to know which leaf node to go to so it is faster.

This section reports the findings for **Experiment 4**:

retrieve those movies with the attribute “FG_PCT_home” from 0.6 to 1, both inclusively

```
Experiment 4: retrieve those movies with the attribute FG_PCT_home from 0.6 to 1
- Number of records retrieved: 238
- Running time of Retrieval process: 3.5084e-05 seconds
- Number of index nodes the process accesses: 22
- Average of FG3_PCT_home values of the records that are returned: 0.50255462
- Number of data blocks the process accesses: 225
- Number of Data Blocks Accessed by Brute Force: 2951
- Running time of Linear Scan Accessed by Brute Force : 0.001317583 seconds
```

Figure 17: Experiment 4 results

Observation

The time complexity of **searching values in a range in B+ Tree is also $O(\log n + k)$** , where n is the number of nodes in the tree and k is the number of values in the range.

It takes $O(\log n)$ to traverse through all levels in the tree and the number of keys in leaf nodes to iterate is proportional to the number of values within range so time complexity is $O(k) + O(\log n)$.

Similarly, B+ Tree retrieval process is **significantly faster than brute force** as brute force has to iterate through all data blocks and each record in data blocks.

3.5. Experiment 5: Records Deletion

This section reports the findings for **Experiment 5**:

delete those movies with the attribute “FG_PCT_home” below 0.35 inclusively, update the B+ tree accordingly.

```
Experiment 5: delete those movies with the attribute "FG_PCT_home" below 0.35 inclusively
- Number of Nodes of Updated B+ Tree: 2389
- Number of Levels of Updated B+ Tree: 4
- Content of root node: (0.402, 16) (0.426, 2) (0.444, 6) (0.463, 3) (0.481, 36) (0.5, 135) (0.524, 23)
- Running time of Process: 0.000135 seconds
- Number of Data Blocks Accessed by Brute Force: 2951
- Running time of Linear Scan Accessed by Brute Force: 0.00188 seconds
```

Figure 18: Experiment 5 Results

Observation

As the leaf nodes in B+ trees are sorted, we are able to identify the desired range of records without searching all nodes. This helps to improve the time of search drastically as compared to brute force searching all nodes.

4. Conclusion

Through this project report, our team successfully managed to build a high-level B+ tree for database storage and indexing. We also compared the run-time analysis of the brute force linear search with our B+ tree search and learnt that a **B+ tree is way more time efficient in terms of search and deletion.**

This is because in a B+ Tree, the number of I/Os to reach the target node containing the lower bound of range search is simply the number of levels = 4. From the lower bound to the upper bound of range search, the additional number of I/Os will simply be the additional number of nodes containing the search keys * 2 (1 I/O for leaf node, and 1 I/O for data block). The number of I/Os access is kept to the minimum as it only access nodes and data blocks that are relevant in the search process.

On the other hand, Brute Force linear search iterates through all data blocks available and loads all the records, only storing the records that are within the search range. Thus, the total number of I/Os = the total number of data blocks in storage = 2951. Thus the number of I/Os is very large as it consists of many block accesses that are irrelevant in the search process, resulting in the running time to be consistently larger than B+ Tree search.

In summary, B+ Tree is superior to Brute Force linear scan in terms of

- Search Efficiency:
 - B+Tree : $O(\log n) + O(k)$
 - Brute Force : $O(n)$
- I/O optimisation
 - B+ Tree: #I/Os directly related to height of B+ Tree, thus efficient for large datasets
 - Brute Force: Scan all data blocks, thus large and fixed #I/Os
- Range Queries
 - B+ Tree more efficient at range queries due to the above-mentioned reasons

5. Appendices

5.1. Project Setup

Please refer to the [README.md](#) in the source code for information on setting up.

Project Setup [↗](#)

Perform the following steps to setup the project.

Our codebase have successfully ran in both Windows & Mac OS. NOTE: Please ensure that **CMake** is properly installed on your device.

1. Create a directory for the build files.

```
mkdir cmake_build
```
2. Change directory to the new directory.

```
cd cmake_build/
```
3. CMake the project using the CMake.txt requirements file.

```
cmake ..
```
4. CMake build to create the executable file.

```
cmake --build . --config Release
```
5. Run the executable file and observe the output in your IDE terminal.

```
cmake ./SC3020_Database_System_Principles
```

Alternative Setup [↗](#)

1. Build the project using your IDE CMake, specifying the `./CMakeLists.txt`
2. Run the project in your IDE.
3. View the output in the terminal.

Figure: Project Setup

5.2. Helper Functions

Memory Pool

Function	Definition
allocateBlock()	Allocates memory space for a block within the memory pool (BLOCK_SIZE = 400).
allocateRecord()	Allocates memory space for the data record within the memory pool (RECORD_SIZE = 40).
saveRecord(Record newRecord)	Saves the data record in the memory pool given the Record structure, calls allocateBlock() whenever there is insufficient memory space in the current block.
saveBPlusTreeNode(BPlusTreeNode *newNode)	Saves the B+ tree node in the memory pool given the BPlusTreeNode structure, calls allocateBlock() whenever there is insufficient memory space in the current block.
saveBPlusTree(BPlusTree tree)	Saves the B+ tree in the memory pool given the BPlusTree structure, saved at the end of the memory pool.

B+ Tree Insertion

Function	Definition
insertKey(MemoryPool *disk, float key, void *recordAddress)	Main function for inserting key and corresponding record address, and stores created B+ tree node into memory pool.
searchInsertionNode(float key)	Search the node for insertion given the float FG_PCT_home value.
getInsertionBPKey(BPlusTreeNode *target, float key)	Gets the unique B+ tree key for insertion, given the float FG_PCT_home, by checking the tree for duplicates and incrementing the count.
split(BPlusTreeNode *cur, BPlusTreeKey bpKey, void *address)	Handles the splitting of a given node, which creates a new B+ tree node, this works for both non-leaf and leaf nodes. The resulting two nodes have 11 nodes each, since $m = 22$.
propagate(MemoryPool *disk, BPlusTreeNode *cur, BPlusTreeKey bpKey, void *address)	Recursive function which upward propagates the insertions in the leaf node to update the parent nodes. It involves splitting the full current node, creating a new root node if necessary, for the current and new nodes, and updating parent nodes with the lower bound of the right subtree.
getSmallestRightSubtree(BPlusTreeNode *node)	Retrievers the smallest FG_PCT_home value of the right subtree given a BPlus tree node.
swapTemp(BPlusTreeNode *node, int index, BPlusTreeKey *temp, void **temp_address)	Swaps the key and memory address of the temporary B+ tree key with the B+ tree key in the current node, based on the index provided.
swapNonLeafTemp(BPlusTreeNode *node, int index, BPlusTreeKey *temp, void **temp_address)	Swaps the key and memory address of the temporary B+ tree key with the B+ tree key in the current node, based on the index provided, which refers to index + 1 position.

B+ Tree Search

Function	Definition
searchKey(MemoryPool *disk, float lowerkey, float upperkey)	searchKey function is created to search for the leaf nodes that contain keys that fall within the lower and upper bound of the specified "FG_PCT_home" value and produce SearchAddressList that contains a list of record addresses retrieved from children nodes of the leaf nodes. It will also count the number of index nodes accessed and store in indexblks
getAverage(MemoryPool *disk)	getAverage function iterates through each record address stored in SearchAddressList to retrieve the "FG3_PCT_home" value from the record and compute the total values to obtain an average.
getNumDataBlock(MemoryPool *disk)	This function determines the number of unique data blocks that are used to store the records with keys that fall within the range of "FG_PCT_home". The retrieval of the records in SearchAddresslist is done via searching through the B+ tree described in the searchKey function.
getBlocksAccessedByBruteForce(float lowerkey, float upperkey)	A function called allocateRecord is used to allocate memory to store the data records in blocks and for each block that is allocated to store the records, its starting address is stored in a vector called block_ptr_list. This function getBlocksAccessedByBruteForce will scan

	every block pointed by block_ptr_list and within each block, scan every record to identify those with key value that falls within the range of FG_PCT_home.
--	---

B+ Tree Deletion

Function	Definition
checkKey(BPlusTreeNode *node)	Upward recursive checks on a node's parents, to ensure that the keys in parent nodes are updated with the lower bound of its right subtree
findLB_rightSubTree(BPlusTreeNode *node, int index_key)	Given a node and the index of a key in the node, it identifies the smallest key in the right subtree of the key's right children
deleteKeyinNonLeafNode()	Deletes the first key and children of a node by shifting the remaining keys & children forward, and reduces the size of node by 1
deleteKeyInLeafNode(MemoryPool *disk)	For Leaf node, the records in disk are also deleted correspondingly
deleteBPlusTreeNode(BPlusTreeNode *node)	Deletes the given node and reduce memory size and number used blocks correspondingly
findNextNonLeafNode(BPlusTreeNode *node)	Given a non leaf node, function returns the right non leaf node
findIndexChild(BPlusTreeNode *childNode)	Returns the index of a child node with respect to the parent non leaf node
MergeWithRight_LeafNode(int num_keys_merge, BPlusTreeNode *leftNode, BPlusTreeNode *rightNode)	Shift current keys and children in right node to the back and iteratively copy the keys and children from left node to the right node. Update size of right node and delete the left node once done
MergeWithRight_NonLeafNode(int num_keys_merge, BPlusTreeNode *leftNode, BPlusTreeNode *rightNode)	For Non Leaf node, a new key is created between the keys copied from left node and the remaining keys in the right node, and the value takes on the lower bound of its right children.
BorrowFromRight_LeafNode(int num_keys_borrow, BPlusTreeNode *leftNode, BPlusTreeNode *rightNode, MemoryPool *disk)	Copies the key and children from right node to left node, and deletes them from right node. Update size of left node once done.
BorrowFromRight_NonLeafNode(int num_keys_borrow, BPlusTreeNode *leftNode, BPlusTreeNode *rightNode)	For Non Leaf node, a new key is created between the original keys and new keys copied over in left node, and the value takes on the lower bound of its right children.